Simplified Meet-in-the-middle Preimage Attacks on AES-based Hashing

Mathieu Degré, Patrick Derbez and André Schrottenloher

Univ Rennes, Inria, CNRS, IRISA, Rennes, France

Abstract. The meet-in-the-middle (MITM) attack is a powerful cryptanalytic technique leveraging time-memory tradeoffs to break cryptographic primitives. Initially introduced for block cipher cryptanalysis, it has since been extended to hash functions, particularly preimage attacks on AES-based compression functions. Over the years, various enhancements such as superposition MITM (Bao et al., CRYPTO 2022) and bidirectional propagations have significantly improved MITM attacks, but at the cost of increasing complexity of automated search models. In this work, we propose a unified mixed integer linear programming (MILP) model designed to improve the search for optimal pre-image MITM attacks against `AES`-based compression functions. Our model generalizes previous approaches by simplifying both the modeling and the corresponding attack algorithm. In particular, it ensures that all identified attacks are valid. The results demonstrate that our framework not only recovers known attacks on `AES` and `Whirlpool` but also discovers new attacks with lower memory complexities, and new quantum attacks.

1 Introduction

The meet-in-the-middle (MITM) attack is a powerful technique using a time-memory tradeoff. Initially introduced by Diffie and Hellman in the cryptanalysis of block ciphers [DH77], it has since then been applied in the cryptanalysis of hash functions [Sas11] and stream ciphers.

In general, the MITM attack aims at solving the system of equations arising from a *constrained path*, be it a block cipher of which a plaintext-ciphertext pair is known, or a compression function given a target preimage. The MITM attack solves this problem through a divide-and-conquer algorithm by first exhaustively and independently solving two sub-systems of equations, usually referred to as sub-paths or *neutral sets*, establishing two *lists* of partial solutions that are merged and sieved using the remaining equations.

Since the first MITM attacks, many subsequent works have increased the freedom in the way the sub-paths were defined, separated and merged. Such techniques include guess-and-determine [DSP07], splice-and-cut [AS08, GLRW10], three-subset MITM [BR10, Sas18], and also sieve-in-the-middle [CNV13] and bicliques [KRS12]. While many of these cited works focus on block ciphers, in this paper we are interested in hash functions, and more precisely, compression functions based on the `AES` block cipher and similar SPN designs (so-called "`AES`-based").

Automatic Search of MITM Attacks. Introducing these various techniques has made MITM attacks increasingly powerful but also more complex. For example, key schedules are often sparse and involve only a few non-linear operations, creating dependencies

E-mail: mathieu.degre@inria.fr (Mathieu Degré), patrick.derbez@inria.fr (Patrick Derbez), andre.schrottenloher@inria.fr (André Schrottenloher)

Date of this document: 2025-12-08.

between round-key bits across multiple rounds, which are hard to detect [LWZ16]. Another challenge arises when the system being solved has multiple solutions, but only one is needed to mount an attack. This is exactly the situation in pre-image attacks against compression functions. In such cases, attackers can reduce the complexity by fixing the values of certain variables or by introducing additional constraints. Automated tools partially solve these challenges by allowing for the exploration of a search space that would be infeasible to cover manually.

One of the first tools to automatically search for MITM attacks on AES-based primitives was designed by Bouillaguet *et al.* [BDF11]. Other tools were later designed to search for more *structured* MITM attacks and Demirci-Selçuk MITM attacks on block ciphers [Sas18, DF16, SSD+18]. But in recent years, the topic that attracted the most attention (and progress) is definitely MITM (pseudo)-preimage attacks on `AES`-based compression functions.

In [BDG+19], Bao *et al.* presented new pre-image attacks against all the three versions of `AES` in hashing modes, improving the previous attacks of Sasaki [Sas11]. In particular, they obtained for the first time better attacks when the key was not fully known in both sub-paths. They therefore opened a new research direction and the main questions were how to generalize their attacks and how to exhaustively search for the best ones. At EUROCRYPT 2021, Bao *et al.* used mixed integer linear programming (MILP) to search for such attacks [BDG+21]. Roughly speaking, the model abstracts out the components of the targeted primitive and uses a color scheme to place them in the two neutral sets. The propagations of colors through different operations (S-Box layer, linear layer) are encoded in a set of rules. In MILP, the coloring scheme is defined using Boolean variables, and the propagation rules via linear constraints.

Throughout the next years, a series of papers improved on this first model by tweaking the propagation rules, introducing new techniques which allowed to find better attacks. Initially focused on hashing, the model was extended to key-recovery and collision attacks in [DHS+21], which also introduced the *non-linearly constrained neutral words*, reaching a 10-round pseudo-preimage attack on `AES`-256. The technique of *guess-and-determine* was introduced in [HDS+22] and in parallel in [BGST22]. Next, Bao et al. [BGST22] introduced the "superposition meet-in-the-middle attack" (SMITM). These MILP models were also extended to Sponge-based hash functions [QHD+23, DZQ+24] and Feistel ciphers [HDQ+23]. Later [CGL+24] introduced linearization of S-Boxes, distributed initial structures, and used structural similarities between encryption and key schedule for some designs. Even more recent results on preimages and collisions appeared in [CGL+25].

The development of these MILP models has led to a remarkable number of new attacks on cryptographic primitives, including `AES`, `Whirlpool`, `Haraka`, and `Streebog` [HDS+22], among others. However, a major drawback of this collection of works is the increasing complexity of the attacks found and the models themselves (although more recent works like [CGL+24] try also to simplify the models and reach a more unified framework). As a consequence, for many state-of-the-art attacks, significant human effort is required to understand them, verify their complexities and explicit their algorithms [DHS+21, BGST22].

New Model. In this work, our goal is to form a new MILP modeling which achieves results as good while being easier to describe than previous works, and that outputs simpler attack algorithms than the ones currently known. The idea of the simple model lies in the continuation of [SS22, SS23], which were so far limited to primitives with simple key schedules. We show how to incorporate a complex key schedule like the one of `AES`. As a downside, we need to update the model of [SS23] to model states at the byte level, increasing the number of variables and constraints.

The "simplicity" of the resulting attack algorithms comes from a more transparent

description of the forward and backward paths used in the MITM attack: to put it bluntly, they require less lines of text. Previous works like [DHS+21, BGST22] introduced complex linear constraints on the key bytes and the internal state bytes, to minimize the number of degrees of freedom representing the paths. Writing down these constraints and verifying them can be tedious. In our alternative modeling, we limit these constraints to the bare minimum. As a consequence, when we obtain a result, writing down an attack algorithm and computing its complexity is relatively straightforward, making it more natural than the latest models for pre-image attacks such as [BGST22].

Scope. We show in this paper that our model allows to retrieve results as good as [SS23, SS22, DHS+21, BGST22] for classical and quantum pseudo-preimage attacks on `AES`-based hashing, and in several cases, we improve the time or the memory complexity. However, comparing exactly the scope of the different models is difficult, as they have different search spaces. Indeed, complex models like [BGST22] require additional heuristic constraints to terminate in reasonable time for some attacks. Meanwhile, our model considers a much restricted search space for possible attacks compared to [BGST22]. Therefore, even when our model gives results as good as [BGST22], it may still be that the solution space of [BGST22] contains better solutions that were not found due to the heuristic constraints.

As our model can be seen as an extension of [SS23, SS22], it allows to retrieve previous results of these papers. It also contains the *guess-and-determine* and *distributed initial structures* [CGL+24] techniques (i.e., distributed matching points). It does not include the technique of *nonlinearly constrained neutral words* [DHS+21], but this is on purpose, as we explain later on.

Another technique not included in our model is the linearization of S-boxes [CGL+24]. Indeed, like all previous works before [CGL+24], the neutral words in our model are oblivious to the S-Box layer (a byte known in input of the S-Box remains known in input, and conversely). The most important variables in our model represent the status of columns in an `AES` state, rather than individual bytes. On the contrary, linearization concerns the S-Box layer. Incorporating this technique in our modeling remains therefore an interesting open question.

Results. As shown in Table 1, we are able to retrieve all pseudo-preimage attacks against both `AES` and `Whirlpool` from previous works, except the ones relying on linearization [CGL+24]. Our model allows to reduce the memory of some of these attacks, for example 8-round `AES`-128, and appears particularly suited to search for quantum attacks, allowing us to obtain several new results. The code of our model is available at:

`https://gitlab.inria.fr/capsule/simplified-meet-in-the-middle`

Organization. In Section 2 we recall the `AES` as well as quantum attacks. Then, Section 3 is dedicated to the description of MITM attacks against compression functions according to our point of view; therefore, preliminaries on MITM attacks are deferred in Section 3. Section 4 is dedicated to our new MILP model. Finally, in Section 5 we present several new results obtained with our model.

2 Preliminaries

In this section, we give some preliminaries regarding the `AES` family of block ciphers and quantum computing.

Table 1: Previous and new (pseudo)-preimage attacks on AES-based compression functions. In the quantum setting, the memory is QRAQM (refer to Section 2.2). We indicate in **bold** new attacks or improvements in the complexities.

Target	Rounds	Time	Memory	Quantum	Source
AES-128	8/10	2^{120}	2^{32}	No	[BDG+21]
	8/10	2^{120}	$\mathbf{2^{16}}$	No	Figure 7
	7/10	2^{60}	2^{8}	Yes	[SS23]
	7/10	$\mathbf{2^{56}}$	2^{16}	Yes	Figure 11
AES-128 with fixed key	7/10	2^{120}	2^{8}	No	[Sas11]
	7/10	2^{112}	2^{32}	No	[CGL+25]
	7/10	2^{112}	2^{32}	No	Figure 6
AES-192	9/12	2^{112}	—	No	[BGST22]
	9/12	2^{112}	2^{112}	No	Figure 9
	10/12	2^{124}	2^{124}	No	[CGL+24]
	9/12	$\mathbf{2^{60}}$	2^{24}	Yes	Figure 10
AES-256	10/14	2^{120}	2^{56}	No	[DHS+21]
	10/14	2^{120}	$\mathbf{2^{48}}$	No	Figure 8
	9/14	$\mathbf{2^{60}}$	2^{8}	Yes	Figure 12
Whirlpool	7/10	2^{480}	2^{128}	No	[BGST22]
	7.75 /10	2^{480}	2^{256}	No	[CGL+24]
	6/10	2^{416}	2^{288}	No	[CGL+24]
	6/10	$\mathbf{2^{232}}$	2^{128}	Yes	Figure 13

2.1 Specification of AES

The Advanced Encryption Standard (AES) [DR20] is a block cipher standardized by the National Institute of Standards and Technology (NIST) in 2001 and is certainly the most widely used symmetric encryption scheme. AES operates on 128-bit blocks and supports keys of 128, 192, and 256 bits. The number of encryption rounds varies based on the key size: 10 rounds for AES-128, 12 for AES-192, and 14 for AES-256. The internal state is represented as a matrix of bytes, which are numbered from 0 to 15 as follows:

$$\begin{pmatrix} 0 & 4 & 8 & 12 \\ 1 & 5 & 9 & 13 \\ 2 & 6 & 10 & 14 \\ 3 & 7 & 11 & 15 \end{pmatrix}$$

Each encryption round consists of four core operations:

- **AddRoundKey** (ARK) – A bitwise XOR operation between the state and a round key derived from the original key using a key schedule.
- **SubBytes** (SB) – A non-linear substitution step where each byte in the state is replaced using an S-Box, which provides resistance against differential and linear cryptanalysis.
- **ShiftRows** (SR) – A transposition step where rows of the state matrix are cyclically shifted to introduce diffusion.
- **MixColumns** (MC) – A linear transformation where each column is multiplied by a fixed MDS (Maximum Distance Separable) matrix.

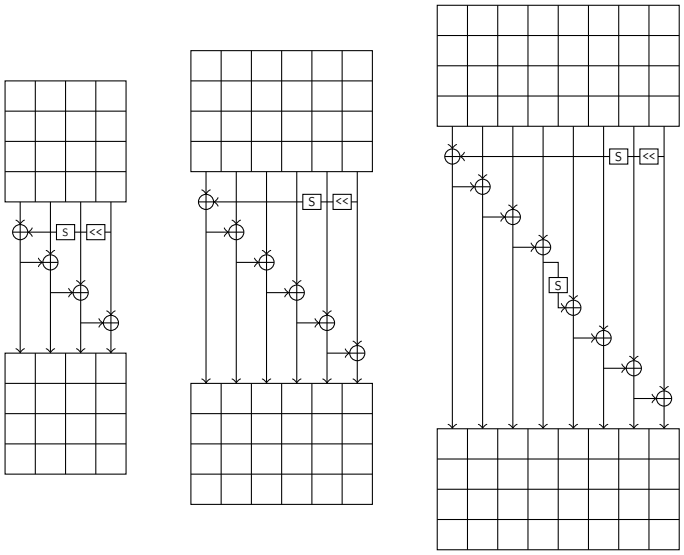


Figure 1: Key schedules of the three versions of `AES` (from TikZ for cryptographers).

The states at round i after ARK, SB, SR and MC are denoted as x_i, y_i, z_i, w_i respectively. We denote as k_i the state of the round key and $u_i = MC^{-1}(k_i)$ an equivalent key obtained by going through MixColumns. We write $x_i[0, 1, 2]$ for a tuple of bytes in a given state, at a given round.

The key schedule is an essential part of `AES`, expanding the secret key into a series of round keys through a combination of byte substitutions, rotations, and XOR operations with round constants (Figure 1).

2.2 Quantum Computing and Quantum Cryptanalysis

Quantum algorithms are known to speed-up many families of attacks. In this paper, we require only a very high-level understanding of quantum algorithms, and will rely on previous works for explicit details of the algorithms involved. A self-contained introduction to quantum computing can be found in [NC02]. Similarly to the analysis of classical attacks, we count the quantum time complexity as a number of computations of the (reduced) `AES`-based primitive. Furthermore, we do not consider parallelization trade-offs.

The main tool used in this paper is Grover's quantum search algorithm [Gro96], which offers a quadratic speed-up of any exhaustive search procedure, and quantum amplitude amplification (QAA) [BHMT02], which similarly speeds up randomized search algorithms. Classical exhaustive search finds preimages on n-bit hash functions in time 2^n, while Grover's search speeds up to a quantum time $\frac{\pi}{2}2^{n/2}$ (in computations of the compression function). A quantum preimage search is a valid attack if it performs better than Grover's search. Since the algorithms that we consider are also based on QAA, they can always be "dequantized" into specific cases of classical MITM attacks. In fact, being a quantum attack acts as an additional constraint on the MITM path.

The most common memory requirement for quantum MITM attacks is quantum memory with quantum random-access (QRAQM). It is a type of memory which stores quantum states (i.e., qubits), while allowing fast random access operations of the form:

$$|x_1, \ldots, x_M\rangle |i\rangle |y\rangle \mapsto |x_1, \ldots, x_M\rangle |i\rangle |y \oplus x_i\rangle \quad .$$

In practice, such a requirement arises because of memory access operations (collision-finding and / or list-merging) which are performed *inside* a Grover's search, making the

whole memory a quantum state. While the QRAQM model is non trivial, its practical validity is, at best, questionable, which also motivates the search for more memory-efficient algorithms.

3 Algorithms

This section depicts how the MITM pseudo-preimage attacks work, both in the classical and quantum setting. We simplify the algorithms and their complexity analysis by assuming that memory access to a large sorted table costs 1, i.e., less or equal to the evaluation of the attacked primitive. In the quantum setting, we make the same assumption for quantum random-access. Note that the MITM attacks described here correspond to those included in our model only.

3.1 Basic Scenario

We consider an `AES` (or variant) block cipher used as a compression function in the Matyas-Meyer-Oseas (MMO) or Davies-Meyer (DM) modes (Figure 2). In the *pseudo-preimage* setting, we control both inputs: the chaining value h and the message block m. Given a fixed target t, we are looking for a pair (h, m) such that $E_h(m) \oplus m = t$ (MMO case) or $E_m(h) \oplus h = t$ (DM case). We let n be the block size of E and κ be its key size, counted in bytes.

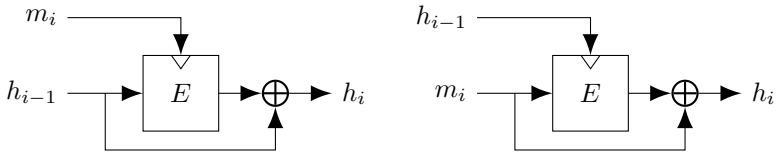


Figure 2: DM (right) and MMO (left) modes (figures from Tikz for Cryptographers).

In the following, by *byte* we will mean a byte of the internal state, or the round key, at a given position, after a given operation of a given round. The primitive is abstracted as a (kind of) undirected graph where the value of a byte can be deduced from the values of other bytes. For example, $y_i[0]$ can be deduced from $x_i[0]$ by SB, while $w_i[0]$ can be deduced from $y_i[0, 1, 2, 3]$ by MC.

Computational Paths. We define a *forward path* and a *backward path*, which are disjoint sets of bytes, either from the key or the internal state, spanning multiple states and rounds, e.g., $(x_0[0], y_0[0], w_0[1], k_1[1] \ldots)$. They are represented by coloring the bytes in blue and red respectively, following the conventions of previous works [Sas11]. As a consequence, we also use the alternative names of *blue path* and *red path*. We display an example using three rounds of `AES` in Figure 3, where the blue path covers half of the states of x_0 to z_1, and the red path covers three quarters of the states in w_2 to w_1.

In the literature, these two sets are also named *neutral sets* [BDG+19]. The reason is that one can compute their values independently from one another. Referring to Figure 3, if we take an arbitrary value for the blue bytes of x_0, then we can compute all blue bytes (going forwards). Likewise, if we take an arbitrary value for the red bytes in w_2, then we can compute all red bytes (going backwards). Note that while the example taken here is simple, an internal state at any round may contain *both* red and blue bytes. In general, we will not draw arrows in our figures, as the order in which bytes are computed depends only on their color.

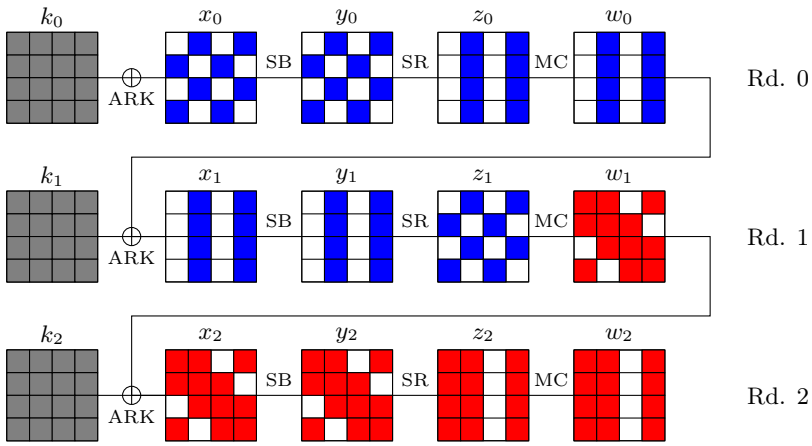


Figure 3: Example of forward (blue) and backward (red) paths, with fixed bytes (grey).

In this example, we also represented the key. For now, its value is fixed (like in [Sas11]). Therefore it will be known both in the forward path and the backward path; such bytes are colored in grey.

Although the forward and backward paths are disjoint, there exist multiple *matching points* throughout the cipher at which we can write equalities between them. For example, if we put $x_0[0]$ in the blue path and $y_0[0]$ in the red path, we know that $y_0[0] = S(x_0[0])$. Non-trivial linear relations are obtained through the MC step. As the matrix used in MC is MDS, we can see on Figure 3 that on each column of z_1 and w_1, since 5 bytes are colored in total, there exists a linear relation between them.

Lists. We let $\mathcal{L}_B$ be the "blue list", or the list of all values that the blue path can take, and $\mathcal{L}_R$ be the "red list" similarly. By "taking values," we mean that the state or key bytes are evaluated, but any relations between them should also be taken into account at this point. That is, if $(x_0[0], y_0[0])$ are both in the blue path, the pair can only take 2^8 values since $y_0[0] = S(x_0[0])$. As an example, the blue list in Figure 3 is of size 2^{64}, as the blue path is determined entirely by the 8 blue bytes of x_0, and the red list is of size 2^{96}, as the red path is determined by the 12 red bytes of w_2.

We assume without loss of generality that $|\mathcal{L}_B| \leq |\mathcal{L}_R|$. The basic classical MITM attack consists in:

1. Compute the entire list $\mathcal{L}_B$, and sort it according to the values at the matching points.
2. For each element in $\mathcal{L}_R$, find matches in $\mathcal{L}_B$.
3. For any match, recompute the entire path and find if it is a pseudo-preimage.

Assuming that $|\mathcal{L}_B| \times |\mathcal{L}_R| = 2^n$, where n is the internal state size, we know that the pairs $(x_B, x_R) \in \mathcal{L}_B \times \mathcal{L}_R$ generate a set of 2^n computational paths, which should contain a solution to the MITM problem (since it is an n-bit constraint). However, we also know that such a pair should satisfy the relations at the matching points between $\mathcal{L}_B$ and $\mathcal{L}_R$. Thus we do not have to check all the pairs, only those that match. We name the list of matching pairs the "merged list" of pairs $\mathcal{L}_M$ and we have $\mathcal{L}_M \subseteq (\mathcal{L}_B \times \mathcal{L}_R)$. Since we consider only linear relations at the matching points, the elements of $\mathcal{L}_M$ can always be enumerated in time $|\mathcal{L}_M|$.

At this point, it should be noted that the entire MITM attack is determined by the choice of the forward and backward paths, i.e., the blue / red coloration of bytes through

all operations of the cipher. This remark is the starting point for the MILP modeling of MITM attacks, which will be covered later in Section 4.

Sub-MITM Problems. We denote the sizes of the lists $\mathcal{L}_B, \mathcal{L}_R$ and $\mathcal{L}_M$ as $2^{8\ell_B}$, $2^{8\ell_R}$, $2^{8\ell_M}$ respectively. Overall, ℓ_B (resp. ℓ_R, ℓ_M) can be approximated by counting the number of blue (resp. red, resp. blue and red bytes) and deducting the number of relations between them. We can hope that $\mathcal{L}_B$, $\mathcal{L}_R$ and $\mathcal{L}_M$ can be computed in time $|\mathcal{L}_B|$, $|\mathcal{L}_R|$, $|\mathcal{L}_M|$ respectively, which will immediately relate the MITM attack's time complexity to the list sizes. However, this is not always true in general. Indeed, the relations inside the blue (resp. red) path can also create kinds of circular dependencies and / or overdefined parts of the path. This problem is identified as *nonlinearly constrained neutral words* in [DHS+21], but more generally, we can view it as a *sub-MITM* problem. That is, computing $\mathcal{L}_B$ and $\mathcal{L}_R$ is generally another MITM problem which would require another MITM strategy. In [DHS+21] the authors gave a generic (but memory-heavy) table-based approach. Our strategy is instead to ensure that the sub-MITM problems are always trivial to solve.

3.2 Grey Variables and Computing the Paths

In pseudo-preimage attacks, the system is highly underdetermined: even if the entire key were fixed, one would expect one solution on average. Thus, one may add arbitrary equations and still have a solution. In addition to the red / blue colors of bytes in the path, we use a *grey* color for bytes whose value will be arbitrarily fixed (similarly to [Sas11] and subsequent works). Grey bytes are known in both the blue and red paths.

Virtual Bytes. In our work, the explicit grey color will only be used in the key path (like in Figure 3), while other works also used explicit grey bytes in the state path (for example, [BDG+19]). This is because in the state, there is an overall better strategy: instead of fixing the value of bytes, we fix a linear combination of them happening at a MixColumns operation.

Definition 1. A *virtual byte* at round i is a linear combination of bytes of states before MC or after MC. These bytes are either:

- $z_i[j] \oplus u_{i+1}[j]$ and $w_i[j]$ (case when the key addition is moved before MC)
- $z_i[j]$ and $x_{i+1}[j] \oplus k_{i+1}[j]$ (case when the key addition is after MC)

Consider for now a path without blue or red key bytes, as in Figure 3, which allows to replace x_{i+1} by w_i. On a given column of the states z_i, w_i (before and after MC at round i), when $c > 4$ bytes are colored (i.e., known in the red or blue paths), it is always possible to write $c-4$ linear equations between them. Furthermore, it is always possible to separate the blue and red bytes in these equations. Consider for example the first column of z_1, w_1 in Figure 3. We know that there exists a linear equation between $z_1[1,3]$ and $w_1[0,1,3]$. Since our intention is to separate the blue and red paths, we can rewrite it as:

$$L_1(z_1[1,3]) = L_2(w_1[0,1,3]) \ .$$

where L_1, L_2 are two linear functions. We can therefore introduce a *virtual byte* t such that:

$$t = L_1(z_1[1,3]) = L_2(w_1[0,1,3]) \ .$$

Fixing the value of t makes it into a *virtual grey byte*. It would be tedious to write down the definitions of all virtual bytes, and to represent them explicitly on a figure such as Figure 3. Fortunately, only the *number* of virtual grey bytes for each column needs to

be known to determine the time complexity of our attack. This number will be represented above each column.

In previous models, virtual grey bytes are defined implicitly in [BDG+21] and explicitly in [SS22].

In the example, if we do not set the virtual variables as grey, then we will instead have a matching point. Indeed, the role of virtual grey bytes is to *replace matching points*. They will reduce the size of the blue and red lists, and reduce the time and memory complexity of the full attack. There is no limit to the amount of matching points that can be replaced by virtual grey variables, however:

- Only κ grey bytes (corresponding to the expected number of solutions to the MITM problem) are free. If more bytes (virtual or not) are colored grey, we will need to iterate upon all possible values for these bytes, before expecting a solution. These grey bytes are respectively named *global constants* and *episodic constants* in previous works [DHS+21].

- If too many virtual grey bytes are set, the blue (resp. red) path become over-constrained, leading to sub-MITM problems.

To see this, let us start from the situation without grey bytes. The only equations between bytes of the blue path are those arising from the cipher's propagation. It is quite easy that each element of the blue list (i.e., valuation of the blue bytes) can be computed in time 1, by taking arbitrary guesses for the free variables at each round, and deducing the other ones from the previous round. However, introducing grey and virtual grey variables creates additional equations on these bytes. At the limit, we may have an entire path spanning multiple rounds but having only 1 solution on average due to these constraints, which is a MITM problem. To avoid this, we adapt the strategy [SS22], that constraints how many virtual bytes can be colored grey. In the remaining of this section, we explicit these constraints.

Constraints on the Keys. One of the main simplifications of our model compared to previous works like [BDG+21] is that we do not define virtual bytes in the key path. Instead, each byte of the round keys is colored in blue, red or grey (and later green). The only non-trivial constraint on round keys that we authorize is a *cancelation* constraint. At a given byte, a *cancelation* will remove the blue or the red color. A cancelation is a non-trivial equation on the key bytes which must be satisfied by the forward (resp. backward) path, and which may be linear or nonlinear. We will not need to write explicitly these equations, as explained below.

We assume that the values of grey bytes in the key path are fixed for the entire attack. This is always possible since we have at most κ constraints on the key (after which the entire key is fixed), and we have the right to fix κ bytes in total.

For the blue (resp. red) path, we will fix a *starting round* at which a given number of bytes are colored. Let $\mathsf{sk_B}$ (resp. $\mathsf{sk_R}$) be the number of colored bytes in the starting round, and $\mathsf{c_B}$ (resp. $\mathsf{c_R}$) be the number of cancelations in other rounds. Then, the number of blue (resp.) red key paths is:

$$|\mathcal{K}_B| = 2^{8(\mathsf{sk_B}-\mathsf{c_B})}, \qquad |\mathcal{K}_R| = 2^{8(\mathsf{sk_R}-\mathsf{c_R})} \ .$$

This might be an underestimate, because some cancelations may be redundant (i.e., fixing several bytes to grey might allow us to deduce another grey byte). In that case we simply expect the algorithm to return exactly $|\mathcal{K}_B|$ (resp. $|\mathcal{K}_R|$) paths, even if more exist. Furthermore, we can compute the list $\mathcal{K}_B$ (resp. $\mathcal{K}_R$) once for the full attack, and store it in memory.

Constraints on the States. Assuming that the key path is now fixed, we reuse constraints from [SS22] on the amount of virtual grey bytes, which ensure the existence of a set of $\mathsf{s_B}$ "free" bytes in the blue list, such that $|\mathcal{L}_B| = 2^{8\mathsf{s_B}}$, and the blue path can be entirely deduced in time 1 after guessing the value of these $\mathsf{s_B}$ bytes (same for the red list).

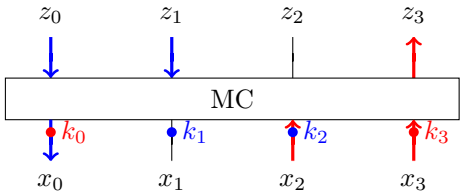


Figure 4: Example of propagation through MixColumns.

Virtual bytes (i.e., matching equations) need only to be defined if both blue and red bytes appear around a MC operation, whether in input, output or in the key. Such a case has been represented in Figure 4, where the z_i are the bytes before MC and x_i are the bytes at the next round, after MC and ARK. We can write two linear equations of the form:

$$\begin{cases} L(z_0, z_1, z_3, x_0 \oplus k_0, x_2 \oplus k_2, x_3 \oplus k_3) = 0 \\ L'(z_0, z_1, z_3, x_0 \oplus k_0, x_2 \oplus k_2, x_3 \oplus k_3) = 0 \end{cases} \tag{1}$$

Since L and L' are linear, we can separate the parts depending on the keys and the states:

$$\begin{cases} L(z_0, z_1, z_3, x_0, x_2, x_3) \oplus L(0, 0, 0, k_0, k_2, k_3) = 0 \\ L'(z_0, z_1, z_3, x_0, x_2, x_3) \oplus L'(0, 0, 0, k_0, k_2, k_3) = 0 \end{cases} \tag{2}$$

and finally, the blue and red parts:

$$\begin{cases} L(z_0, z_1, 0, x_0, 0, 0) \oplus L(0, 0, 0, 0, k_2, 0) = L(0, 0, z_3, 0, x_2, x_3) \oplus L(0, 0, 0, k_0, 0, k_3) \\ L'(z_0, z_1, 0, x_0, 0, 0) \oplus L'(0, 0, 0, 0, k_2, 0) = L'(0, 0, z_3, 0, x_2, x_3) \oplus L'(0, 0, 0, k_0, 0, k_3) \end{cases}$$

At first sight, we could make all virtual bytes grey. However this would constrain significantly the blue and red paths, making their computation a non-trivial sub-MITM problem. A proper limitation of the quantity of grey virtual bytes ensures that the blue and red path are computed trivially. Recall that the blue path is computed *forwards*, where values for the blue bytes at round $i+1$ are deduced from values at round i, and the red path is computed *backwards*. We take the following constraint from [SS22].

Constraint 1. *In any column at round i, there should be less virtual grey bytes than the number of blue bytes in w_i (after MC), and less than the number of red bytes in z_i (before MC).*

Under this constraint, consider a column of z_i and w_i at any round i. Fix the values of the blue bytes in z_i. Then the system of linear equations that relate the blue bytes of z_i, w_i and the virtual bytes is undetermined. This means that we can deduce the blue bytes of w_i immediately, possibly by making guesses of new free variables (which corresponds to the *guess-and-determine* technique). By symmetry, the same is true for the red path. On the example of Figure 4, this means that we would only use one virtual grey byte, fixed to a value t:

$$\begin{cases} L(z_0, z_1, 0, x_0, 0, 0) \oplus L(0, 0, 0, 0, k_2, 0) = t \\ L(0, 0, z_3, 0, x_2, x_3) \oplus L(0, 0, 0, k_0, 0, k_3) = t \end{cases} \tag{3}$$

When going forwards, the byte x_0 is deduced from (z_0, z_1, k_2) using the first equation. When going backwards, the byte z_3 is deduced from (x_2, x_3, k_0, k_3) using the second

equation. The remaining equation may be used as a matching point. As noticed above, the placement of this virtual grey byte can remain implicit when analyzing the algorithm (we could equivalently place it on the first or the second equation).

3.3 Classical Attack

If the aforementioned constraints are set, we can deduce a generic MITM algorithm (Algorithm 1). We use the following definitions and notation:

- $\mathsf{d_B}$ / $\mathsf{d_R}$ / $\mathsf{d_{BR}}$ is the total *dimension* of the blue / red / merged state path (counting only relations between them, and not counting virtual grey constraints);
- $\mathsf{sk_B}$ / $\mathsf{sk_R}$ is the number of blue / red bytes in the starting key state;
- $\mathsf{c_B}$ / $\mathsf{c_R}$ is the number of cancelations of blue / red key bytes;
- v is the number of virtual grey bytes;
- The *dimension* of the blue / red key is defined as the number of byte values which allow to determine it entirely. As shown in Section 3.2 is it lower or equal to $\mathsf{sk_B} - \mathsf{c_B}$ and $\mathsf{sk_R} - \mathsf{c_R}$ respectively.

The algorithm runs multiple *episodes* in which all grey bytes are fixed. The total number of grey bytes is:

$$g := \underbrace{\mathsf{c_B} + \mathsf{c_R} + (\kappa - \mathsf{sk_B} - \mathsf{sk_R})}_{\text{Key}} + \underbrace{v}_{\text{State}} \leq n + \kappa \ .$$

During a MITM episode, the sizes of the lists are:

$$\begin{cases} |\mathcal{L}_B| := 2^{\ell_B} = 2^{8(\mathsf{d_B} - v + (\mathsf{sk_B} - \mathsf{c_B}))} \\ |\mathcal{L}_R| := 2^{\ell_R} = 2^{8(\mathsf{d_R} - v + (\mathsf{sk_R} - \mathsf{c_R}))} \\ |\mathcal{L}_M| := 2^{\ell_M} = 2^{8(\mathsf{d_{BR}} - v + (\mathsf{sk_B} - \mathsf{c_B}) + (\mathsf{sk_R} - \mathsf{c_R}))} \end{cases} . \tag{4}$$

The number of episodes is:

$$|E| := 2^e = 2^{8\max(g-\kappa,0)} = 2^{8\max(v + \mathsf{c_B} + \mathsf{c_R} - \mathsf{sk_B} - \mathsf{sk_R}, 0)} \ . \tag{5}$$

Typically more virtual grey bytes and cancelations will help reduce the list sizes, while they increase the number of episodes.

The time complexity exponent of Algorithm 1 (relatively to the byte size) is:

$$\max\left(\mathsf{sk_B}, \mathsf{sk_R}, e + \ell_B, e + \ell_R, e + \ell_M\right) \tag{6}$$

and its memory complexity exponent is:

$$\max\left(\mathsf{sk_B} - \mathsf{c_B}, \mathsf{sk_R} - \mathsf{c_R}, \min(\ell_B, \ell_R)\right) \tag{7}$$

Indeed, we can store either one of the lists $\mathcal{L}_B$ and $\mathcal{L}_R$, and $\mathcal{L}_M$ is never stored.

3.4 Quantum Attack

Our attacks in the quantum setting use a translation of Algorithm 1 into a *nested quantum search*. Concretely, each "for all" loop is replaced by a level of quantum search. Finding a solution in the inner levels is a condition of success for the outer levels.

For a detailed analysis, we can refer to [SS22, Appendix A]. Let $\mathcal{L}_B$, $\mathcal{L}_R$, $\mathcal{L}_M$ be the blue, red and merged lists, and E be the number of episodes.

Algorithm 1 MITM attack without sub-MITM. The roles of red and blue path can be exchanged.

Input: a target
Output: a pseudo-preimage

1: Fix grey key bytes, and fix virtual grey bytes, up to a total of κ bytes, to arbitrary constants (e.g., 0).
2: Compute and store all $2^{8(\mathsf{sk_B}-\mathsf{c_B})}$ values for the blue key path $\mathcal{K}_B$
3: Compute and store all $2^{8(\mathsf{sk_R}-\mathsf{c_R})}$ values for the red key path $\mathcal{K}_R$
4: **for all** Episodes **do**
5: Fix all other virtual grey bytes
6: **for all** Values of $\mathbf{k}_B$ in $\mathcal{K}_B$ **do**
7: **for all** $(\mathsf{d_B} - v)$ free bytes in the blue path **do**
8: Compute the blue path forwards, round by round, deterministically using the values of the free bytes and the virtual grey bytes
9: **end for**
10: **end for**
Sort $\mathcal{L}_B$ using the values at the unused matching equations
11: **for all** Values of $\mathbf{k}_R$ in $\mathcal{K}_R$ **do**
12: **for all** $(\mathsf{d_R} - v)$ free bytes in the red path **do**
13: Compute the red path, round by round
14: Compute the values for all matching equations
15: **for all** Matching blue paths in $\mathcal{L}_B$ **do**
16: Deduce an entire state and key; recompute the path
17: If the target is met, **return** the value of the state and key.
18: **end for**
19: **end for**
20: **end for**
21: **end for**

Due to the structure of Algorithm 1, the **for** loops at Steps 4, 6, 7, 11, 12 have all a fixed number of iterates. Only the **for** loop at Step 15 has a variable number of iterates. Notably if $|\mathcal{L}_M| \geq |\mathcal{L}_R|$, each element of $\mathcal{L}_R$ sampled is expected to yield several partial solutions. This motivates the following heuristic.

Heuristic 1. *It is assumed that a single solution to the MITM problem exists; the corresponding episode and element of $\mathcal{L}_R$ are selected uniformly at random from all possibilities.*

This allows to bound the expected quantity $|\mathcal{L}_M|/|\mathcal{L}_R|$, and leads to:

Theorem 1 (Adaptation of Theorem 4 in [SS22])**.** *Under Heuristic 1, there is a quantum MITM attack of time complexity:*

$$\frac{\pi}{2}\left(\frac{|\mathcal{K}_B|}{2^{4c_B}}+\frac{|\mathcal{K}_R|}{2^{4c_R}}+\sqrt{|E|}\left(|\mathcal{L}_B|+\left(\frac{\pi}{4}\sqrt{|L_R|}+1\right)\left(\frac{\pi}{\sqrt{2}}\max\left(1,\sqrt{\frac{|\mathcal{L}_M|}{|\mathcal{L}_R|}}\right)\right)\right)\right) \quad , \tag{8}$$

succeeding with probability at least $1/2$, and memory complexity:

$$\frac{|\mathcal{K}_B|}{2^{8c_B}}+\frac{|\mathcal{K}_R|}{2^{8c_R}}+|\mathcal{L}_B| \ . \tag{9}$$

Equation 8 stems from Theorem 4 in [SS22], where we have added the terms for constructing the lists of blue and red key paths (they are found by repeated instances of

Grover's search). Note that blue and red paths are interchangeable in this formula. If we simplify the constant factors, the resulting time and memory complexity exponents are:

$$\max\left(\mathsf{sk_B} - \frac{\mathsf{c_B}}{2}, \mathsf{sk_R} - \frac{\mathsf{c_R}}{2}, \frac{e}{2} + \left(\min(\ell_B, \ell_R) + \max\left(\frac{\ell_R}{2}, \frac{\ell_B}{2}, \frac{\ell_M}{2}\right)\right)\right) \tag{10}$$

and

$$\max\left(\mathsf{sk_B} - \mathsf{c_B}, \mathsf{sk_R} - \mathsf{c_R}, \min(\ell_B, \ell_R)\right) \quad . \tag{11}$$

Therefore, if the terms $\mathsf{sk_B}$ and $\mathsf{sk_R}$ do not dominate in the complexity (i.e., there are few active key bytes), and if $2\min(\ell_B, \ell_R) \leq \max(\ell_B, \ell_R, \ell_M)$, then a classical MITM attack path is also a valid quantum one, and we get a quadratic speedup.

4 Description of the Model

Our MILP model can be seen as an intermediate between the models of [SS22, SS23] and [BDG+21, DHS+21, BGST22]. Like in [SS22, SS23], we count the number of virtual grey bytes and set constraints (notably Constraint 1) that ensure a correspondence between solutions of the model and algorithms. However, handling `AES`-like key schedules forces us to adopt a byte-level modeling of the states, which is closer to [BDG+21, DHS+21, BGST22]. The modeling of the key is, however, much more restrictive than these works: as detailed in Section 3, we authorize only *cancelations* in the round keys, instead of more general linear constraints.

The MILP model uses Boolean variables to represent the colors of state bytes of x_i, y_i, z_i, w_i and key bytes of k_i, u_i at all rounds i. There are four main colors used: **red** describes the backward path, **blue** the forward path, **grey** means a value that is fixed before the merging process (so it is known in both paths); **white** means a value that is not known in any path. Given a coloring scheme for the states and keys, we can deduce the time and memory complexity of the attack; thus these are the main variables of the model. We will then optimize primarily for the time, and secondarily for the memory.

4.1 Modeling the State

For each state byte i, we create two Boolean variables $\mathsf{BlueCol}[i]$ and $\mathsf{RedCol}[i]$ with the constraint $\mathsf{BlueCol}[i] + \mathsf{RedCol}[i] \leq 1$. The variable $\mathsf{BlueCol}[i]$ is 1 if the byte is colored blue, 1 if colored red. If none of them are 1, the byte is not known in either path, and colored white in the figures (see Table 2). Contrary to [BDG+21] and the following works, there is no color "grey" for state bytes, as it is subsumed by the counting of "virtual" grey bytes.

Table 2: Color correspondence of state bytes.

RedCol	BlueCol	Color	RedCol	BlueCol	Color	RedCol	BlueCol	Color
0	0	White	0	1	Blue	1	0	Red

Basic Modeling. As it was already noticed in previous works, additions of constants and application of S-Boxes do not change the colors of individual bytes. Permutation of bytes (e.g., ShiftRows) can be easily implemented as a permutation of variables. Essentially, the only important operation to focus on is MixColumns.

For each column c, we will introduce three Boolean variables $\mathsf{ActiveBlue}[c]$, $\mathsf{ActiveRed}[c]$, $\mathsf{ActiveMerged}[c]$, three integer variables $\mathsf{BGain}[c], \mathsf{RGain}[c], \mathsf{MGain}[c]$ and an integer variable

$\mathsf{virtualgrey}[c]$. Let $b_{\text{in}}, b_{\text{out}}, r_{\text{in}}, r_{\text{out}}$ be the number of blue (resp. red) bytes before and after MC, then the constraints are:

$$\begin{aligned}
\mathsf{BGain}[c] &\leq \min(b_{\text{in}} + b_{\text{out}} - 4 \times \mathsf{ActiveBlue}[c], 4 \times \mathsf{ActiveBlue}[c]) \\
\mathsf{RGain}[c] &\leq \min(r_{\text{in}} + r_{\text{out}} - 4 \times \mathsf{ActiveRed}[c], 4 \times \mathsf{ActiveRed}[c]) \\
\mathsf{ActiveBlue}[c] + \mathsf{ActiveRed}[c] &\leq 1 \\
\mathsf{MGain}[c] &\leq b_{\text{in}} + b_{\text{out}} + r_{\text{in}} + r_{\text{out}} - 4 \times \mathsf{ActiveMerged}[c] \\
\mathsf{MGain}[c] &\leq 4 \times \mathsf{ActiveMerged}[c] \\
\mathsf{virtualgrey}[c] &\leq \mathsf{MGain}[c] - \mathsf{RGain}[c] - \mathsf{BGain}[c] \\
\mathsf{virtualgrey}[c] &\leq \min(b_{\text{out}}, b_{\text{in}})
\end{aligned}$$

These constraints are to be interpreted as follows. $\mathsf{BGain}[c]$ (resp. $\mathsf{RGain}[c]$) is the number of linear relations which can be resolved within the blue (resp. red) path. Obviously this can only happen if at least 4 bytes are blue (resp. red), so both variables are incompatible. $\mathsf{MGain}[c]$ is the number of relations in the merged path, which may include new relations. The constraints on $\mathsf{virtualgrey}[c]$ follow the strategy explained in Section 3.2.

The total *dimension* of the blue (resp. red, merged) state path, i.e., the number of degrees of freedom, is then deduced simply by summing the blue (resp., red, and both) bytes and deducting the corresponding "gain" variables.

Additional Constraints. The constraints given above do not specify at which rounds the computation of the blue (resp. red) path starts, nor at which rounds the matching occurs. This essentially covers the notion of "distributed initial structures" in [CGL+24] (the initial structure being the starting point of the paths). However, doing so requires some constraint which forbid non-trivial paths. First, there needs to be at least one round where all variables RGain (resp. BGain) are zero ("cut round"). Without this constraint, the solver could simply color the entire cipher blue (or red), obtaining a time complexity 0. Second, there needs to be at least one intermediate state that is entirely colored. Indeed, otherwise it is not clear if one can recompute the entire computational path from an element of the merged list, which is a requirement for the MITM attack. Without this constraint, the solver could simply color the entire cipher white, obtaining a time complexity 0.

4.2 Modeling the Key Diffusion

Independently from the internal state, we introduce variables and constraints to model the key bytes. For each key byte i, we create three Boolean variables $\mathsf{BlueCol}[i]$, $\mathsf{RedCol}[i]$, $\mathsf{WhiteCol}[i]$. The byte can either be *grey*, *blue*, *red*, *white* or *green*. This time, the correspondence between variables and colors follows [CGL+24] (Table 3). Grey is the default color, obtained when all Booleans are 0. Blue and red are not exclusive, and having both means that the byte is a linear combination of a blue part and a red part, which is displayed in green on the figures. White means that the value is unknown in both paths; it appears only when a green byte goes through an S-Box.

Having linear combinations of blue and red degrees of freedom is a very restricted variant of the technique of *superposition states* used in more generality in [BGST22].

A general MILP model for the key would allow any coloring of the bytes, and compute at runtime the dimension of the blue and red paths, i.e., the total number of corresponding keys. This appears to be a difficult problem. Previous models allow instead more or less flexibility on the key path, and recompute the dimension of the key afterwards [BDG+21, DHS+21, BGST22]. In our model we choose to restrict significantly the search space of key patterns, in such a way that the dimension of the key, and thus the time complexity of the attack, can be deduced trivially (refer to Section 3.3).

Table 3: Color correspondence of key bytes.

WhiteCol	RedCol	BlueCol	Color	WhiteCol	RedCol	BlueCol	Color
0	0	0	Grey	1	0	0	White
0	0	1	Blue	1	0	1	White
0	1	0	Red	1	1	0	White
0	1	1	Green	1	1	1	White

We will fix a *starting round* for the diffusion of the blue and red bytes in the key. The number of blue bytes (resp. red bytes) in the starting round determines the degree of freedom in the key (i.e., the size of $\mathcal{K}_B$, resp., $\mathcal{K}_R$). We deduce the number of grey bytes in the key.

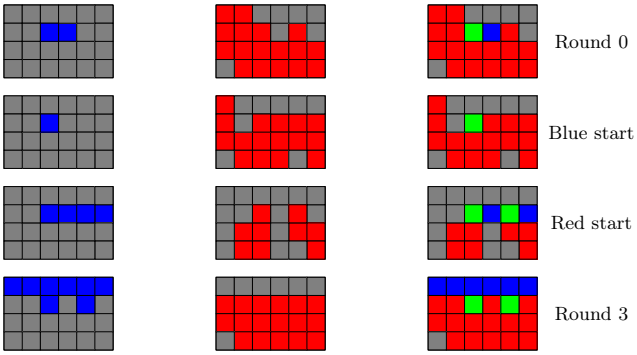


Figure 5: Diffusion of the key in *superposition* between the blue and red paths (the key-schedule used here is the one of AES-192). From left to right: diffusion of the blue key, diffusion of the red key, superposed state.

We express symbolically each byte of the key depending on the bytes of the starting round. These expressions allow us to write constraints for the diffusion of blue (resp. red) bytes, and the appearance of white bytes. To minimize the overlap between blue and red diffusion (and slow down the creation of white bytes), it is useful to start from different rounds for blue and red. This is depicted in Figure 5. Both the blue and red paths (and / or respective blue and red parts in the green bytes) can be computed independently and efficiently. So far we have defined the main key-schedule k, but in attacks against AES-based hashing, we also sometimes consider the equivalent key $u = MC^{-1}(k)$. The colors of u are deduced column by column from the corresponding k.

Cancelations of Key Bytes. As explained in Section 3.2, we introduce cancelations of key bytes, which transform a green byte back into blue or red. The most general setting is to cancel in the key-schedule. For each key byte i, we add two variables $\mathsf{BlueCancel}[i]$ and $\mathsf{RedCancel}[i]$ which can respectively substitute to the colors blue and red. That is, if the byte should be blue (resp. red) due to diffusion, setting the "cancelation" variable to 1 will avoid it. However, it is more efficient to define cancelation variables only at the key addition step. When adding multiple keys at once (like the first and last round key addition in AES variants), a single cancelation variable needs to be set: the linear equation is on the sum of these bytes. Subtle relations between canceled bytes may result in overestimating the colors (i.e., a byte that is actually grey may appear blue in the path), but this does not make the algorithm invalid, and may only lead to overestimating its complexity.

4.3 Interactions Between Key and State

Our model constraints the colors of the key at key additions, similarly to [SS23] (but in an adapted way). The basic idea is that the color of the state before and after ARK constraint the color of the key at this point, in order to allow the computation of the path.

Key Addition. A simple "addition of the key" will take place if we have a sequence of operations of the form: $SB \to ARK \to ARK \to SB$, with one or multiple key additions between two non-linear layers (typically the last round in an AES-based primitive). In that case, we opt for simple byte-wise constraints: the state before and after addition should have the same color, and the keys should match this color or be canceled. (That is, if the state is blue, the key should not contain red, or the red part should be canceled).

Key Addition Around MixColumns. The second way to add the key is when we have a sequence of operations of the form: $SB \to MC \to ARK \to SB$. In that case, the constraints are created column-wise. For each column, we define a Boolean *switch* variable which indicates if we will rather add the key k *after* MixColumns, or the equivalent key $u = MC^{-1}(k)$ *before* MixColumns. This allows to choose between u and k for cancelations, or for the definition of virtual bytes.

Then, we have the following constraints:

- If a byte after MC is blue or red, and the key is k, the corresponding key should not be white;
- If a byte before MC is blue or red, and the key is u, the corresponding key should not be white;
- If BGain is non-zero, the key is k, and a byte after MC is blue, the corresponding key should be non-red (or canceled) (indeed, we need to be able to compute the path);
- If BGain is non-zero, the key is u, and a byte before MC is blue, the corresponding key should be non-red;
- If RGain is non-zero, the key is k, and a byte after MC is red, the corresponding key should be non-blue (or canceled);
- If RGain is non-zero, the key is u, and a byte before MC is red, the corresponding key should be non-blue.

These constraints ensure that the blue and red paths can be computed, i.e., that we do not lose any knowledge of the state after (resp. before) the key addition that happens around MC. No other constraints are necessary for columns at which we *match* between the two paths, because it is always possible to separate them in the linear equations, as noticed in Section 3.2.

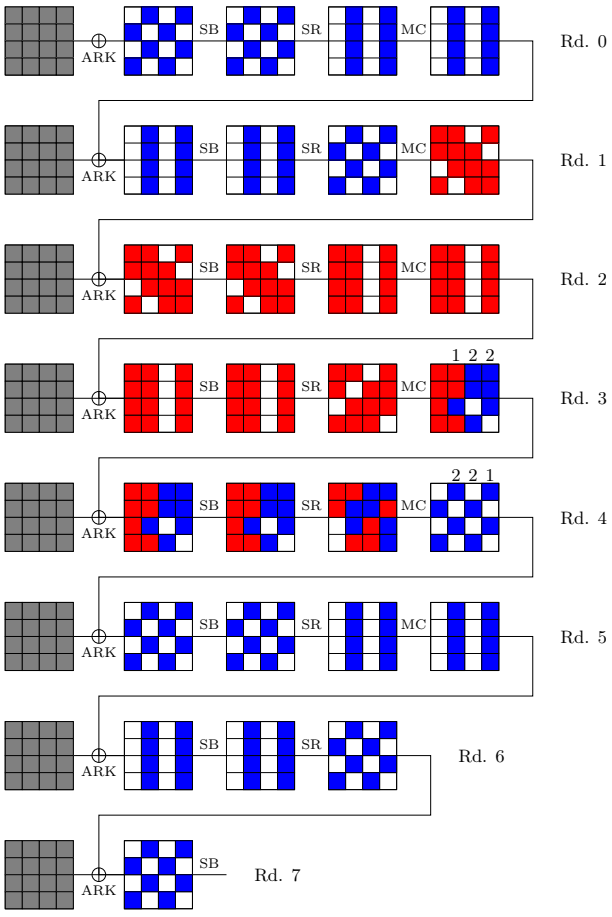
5 Applications: Recovering and Improving Classical Attacks

In this section, we give the path of several new MITM pseudo-preimage attacks on AES-based compression functions (with respectively 128, 192 and 256-bit keys) which recover previous results or improve upon their memory complexities.

The figures represent the direct outcome of our model and the corresponding attack is obtained in a consistent way. First, we generate the key paths, then we write down the equations for matching. Matching always happens around MC, at columns which

have more than 4 colored bytes in input or output. The number of virtual grey bytes, if applicable, is indicated above each column. The blue state path is computed forwards, the red state path is computed backwards. Each time one goes through MC, the list size increases by the number of new blue (resp. red) bytes minus the number of virtual grey bytes. We give more details for each attack. Note that the parameters displayed in each figure are counted in bytes.

5.1 State-of-the-art Attack on 7-round AES-128 with Fixed Key



Blue state: 14, Red state: 14, Blue-red state: 14
Blue key: 0, Red key: 0, Grey key: 16
Blue cancelations: 0, Red cancelations: 0
Time: 14, Memory: 4
Total grey: 26, Virtual grey: 10, Loops: 10
Blue list: 4, Red list: 4, Merged list: 4

Figure 6: Classical pseudo-preimage attack on 7-round AES-128 with fixed key.

As a first example, our model finds a pseudo-preimage attack on 7-round AES-128 with *fixed key* equivalent to [CGL+25], with time complexity 2^{112} and memory complexity 2^{32}. It is displayed in Figure 6.

There are 10 virtual grey bytes, i.e., precomputed linear relations between the blue and red paths: 1+2+2 at round 3 and 2+2+1 at round 4. To be more precise, there are

several positions where the virtual bytes are actually not virtual; that is, we can consider them as fixed bytes in the states. This happens when there are c linear relations, but only c blue (resp. red) bytes in total in these relations:

- In column 1 at round 3 (z_3/w_3), there is a single blue byte: $w_3[6]$;
- In column 1 and 2 at round 4, there are two red bytes: $z_4[4,7]$ and $z_4[10,11]$ respectively;
- In column 3 at round 4, there is a single red byte $z_4[13]$.

After fixing the values of (virtual) grey bytes, the blue, red and merged lists can be computed as follows.

- **Blue list** (forwards): start at round 3. The byte $w_3[6]$ is already known. In column 2 and 3 of w_3, guess two bytes and use the 4 linear relations of virtual bytes to deduce the other blue bytes. Compute forwards until z_4. Guess the values of $w_4[1,3]$. Use the grey bytes $z_4[4,7], z_4[10,11], z_4[13]$ to deduce the other values. Compute forwards until z_1. 4 guesses were made, so the list has size 2^{32}.
- **Red list** (backwards): start at round 4. The bytes $z_4[4,7,10,11,13]$ are grey, thus already known; guess $z_4[0,1]$. Compute backwards until w_3. Use the knowledge of $w_3[10]$ to deduce $z_3[4,6,7]$. In columns 2 and 3, guess one byte and deduce the others with the linear relations. Compute backwards until w_1. 4 guesses were made, so the list has size 2^{32}.
- **Matching:** there are 4 unused linear relations at round 1 (one per column), so the merged list has size $2^{32+32-32} = 2^{32}$.

This gives the time and memory complexities of the attack at $2^{10\times 8+32} = 2^{112}$ and 2^{32} respectively.

5.2 Improved Attack on 8-round AES-128

Our model finds a pseudo-preimage attack on 8-round AES-128 with time complexity 2^{120} (like the state of the art [BDG+21]) and memory complexity 2^{16}, improving on [BDG+21]. The full path is displayed in Figure 7, with the key schedule displayed on the right of the figure. The starting round for the key is round 4, and all bytes except one are fixed.

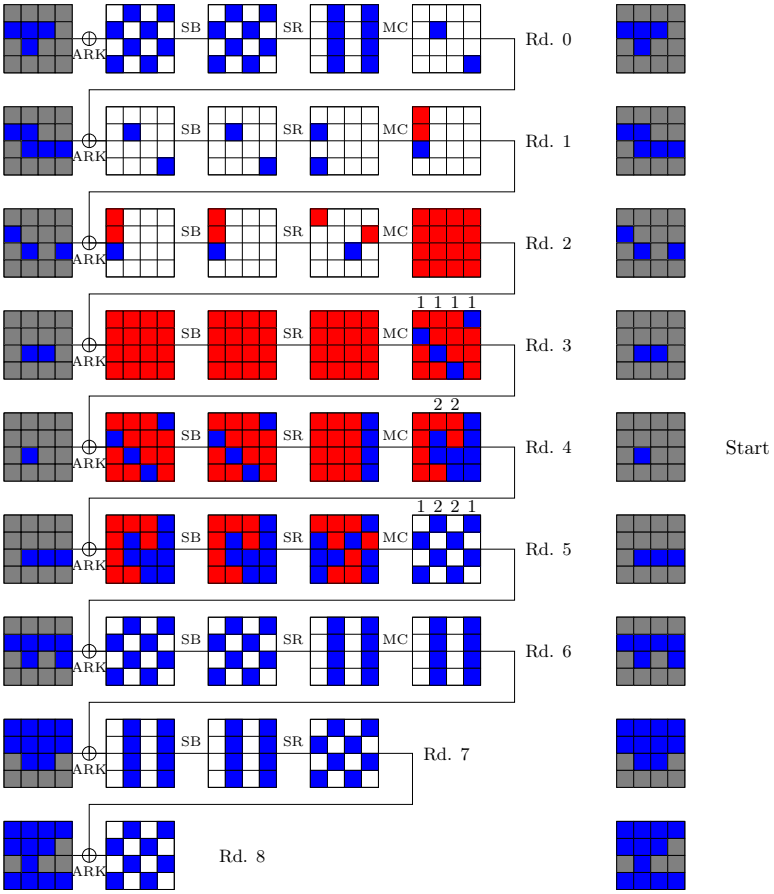
There are 14 "virtual grey" bytes: 4 at round 3, 4 at round 4, 6 at round 5. We detail their definitions.

- **Round 3:** the 3 virtual grey bytes in columns 0, 2 and 3 are actually not virtual, as they correspond to $w_3[1,11,12]$ which can be set as grey. The virtual byte in column 1 is $w_3[6] = x_4[6] \oplus k_4[6]$; there is a linear equation of the form:

$$x_4[6] \oplus k_4[6] = L(z_3[4,5,6,7]) \ . \tag{12}$$

- **Round 4:** In column 1 of w_4 (and similarly for column 2), we can write a system of linear equations of the form:

$$\begin{cases} L_1(w_4[4,7], z_4[4,5,6,7], w_4[5], w_4[6] \oplus k_4[6]) = 0 \\ L_2(w_4[4,7], z_4[4,5,6,7], w_4[5], w_4[6] \oplus k_4[6]) = 0 \\ L_3(w_4[4,7], z_4[4,5,6,7], w_4[5], w_4[6] \oplus k_4[6]) = 0 \\ L_4(w_4[4,7], z_4[4,5,6,7], w_4[5], w_4[6] \oplus k_4[6]) = 0 \ . \end{cases} \tag{13}$$



Blue state: 15, Red state: 16, Blue-red state: 15

Blue key: 1, Red key: 0, Grey key: 15

Blue cancelations: 0, Red cancelations: 0

Time: 15, Memory: 2

Total grey: 29, Virtual grey: 14, Loops: 13

Blue list: 2, Red list: 2, Merged list: 2

Figure 7: Pseudo-preimage attack on 8-round AES-128.

By eliminating $w_4[5]$ and $w_4[6] \oplus k_4[6]$ from the first two equations we get:

$$\begin{cases} L'_1(w_4[4,7], z_4[4,5,6,7]) = 0 \\ L'_2(w_4[4,7], z_4[4,5,6,7]) = 0 \\ L'_3(w_4[4,7], z_4[4,5,6,7]) = t = L''_3(w_4[5], w_4[6] \oplus k_4[6]) \\ L'_4(w_4[4,7], z_4[4,5,6,7]) = t' = L''_4(w_4[5], w_4[6] \oplus k_4[6]) \ . \end{cases} \tag{14}$$

Therefore, there are two linear relations *within the red path* (i.e., between the 6 red bytes in input and output of the MC operation), and two virtual bytes t, t'. These virtual bytes are set as grey in our attack.

- **Round 5:** In column 0 and 3, the virtual bytes are also not virtual: they correspond to the single red byte of each column which will be fixed. In column 1 and 2, there are two independent linear relations between the red and blue bytes, and the virtual bytes are defined accordingly.

The full attack is obtained as follows. We start by setting a value for the 15 grey bytes of the 5th round key which can be propagated to all grey bytes of the round keys. We compute the list of 2^8 possible key paths by varying $k_4[6]$. We then set the value of the 14 virtual grey bytes: one of them will be fixed throughout the entire attack (like the key grey bytes). For all choices of the 13 others, we repeat the following:

1. **Red list:** we compute backwards, starting from z_5. We guess one byte of column 1 and one byte of column 2, and deduce the 4 others using the virtual grey bytes. We compute backwards to w_4. Then, in column 1 and 2, we use the 4 linear equations of Equation 14 (the last two depending on the virtual grey bytes) to deduce z_4. We compute backwards to w_3, then use the grey bytes to compute backwards two more rounds, until $w_1[0,1]$. As we made two guesses, the list has size 2^{16}. It is stored in memory.

2. **Blue list:** we compute forwards, starting from w_3. At round 3 the values of the blue bytes are fixed and no guesses need to be made until z_4. In column 1 and 2 of w_4, we use the value of the grey virtual bytes to deduce the new blue bytes from Equation 14. We compute forwards until z_5. In columns 0 and 3 of z_5, the remaining red byte is actually grey, so we can compute forwards again. In columns 1 and 2, we deduce the two blue bytes of w_5 thanks to the two linear relations. We can then compute forwards until z_1. The only byte that we need to guess is $w_1[2]$. As the blue key path takes 2^8 values, the blue list is of size 2^{16} in total.

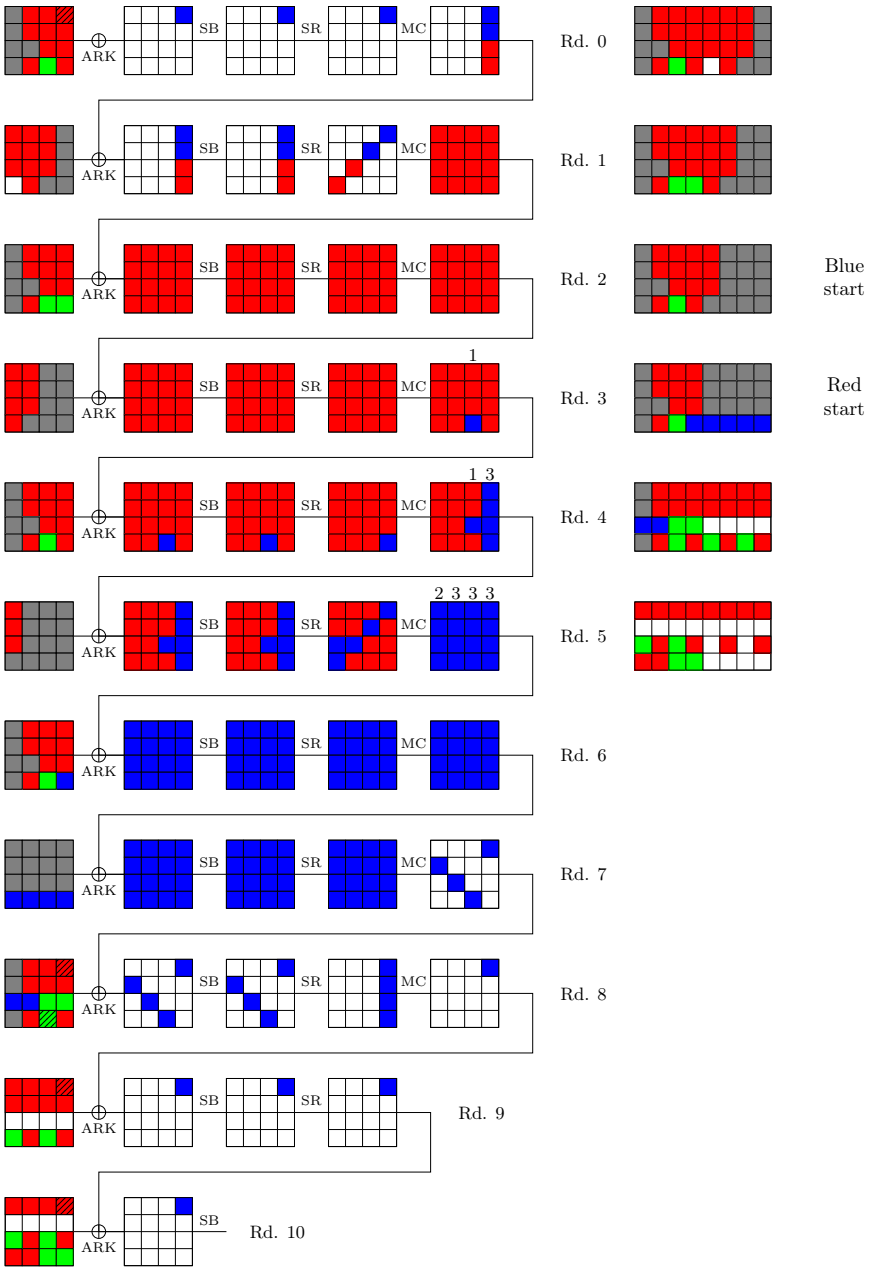
3. **Matching:** The two lists are merged using two linear equations: one in column 0 of z_1, w_1 and one in column 2 of z_2, w_2. Note that in the latter, there is a non-trivial separation between the blue and red parts as $k_3[10]$ is blue. As noticed before, at matching points the color of surrounding key bytes does not matter. We expect the merged list to have size 2^{16}.

After trying $2^{8\times 13} = 2^{104}$ different values for these variable grey bytes, we expect a solution.

5.3 Improved Attack on 10-round AES-256

Our model finds a pseudo-preimage attack on 10-round AES-256 with time complexity 2^{120} and memory complexity 2^{48}, which improves over the one of [DHS+21] (2^{56}). The path is displayed in Figure 8. We detail below the key steps of the attack.

- **Red key paths:** in the starting state for the diffusion of the red key bytes, there are 10 red bytes. Later in the path there are 4 positions for red cancelations, i.e.,



Blue state: 21, Red state: 16, Blue-red state: 15

Blue key: 1, Red key: 10, Grey key: 21

Blue cancelations: 0, Red cancelations: 4

Time: 15, Memory: 6

Total grey: 41, Virtual grey: 16, Loops: 9

Blue list: 6, Red list: 6, Merged list: 6

Figure 8: Pseudo-preimage attack on 10-round AES-256.

4 relations that the red key must satisfy. These positions are indicated by a black pattern in Figure 8: they are $k_0[12] \oplus k_{10}[12]$ (a single cancellation for the XOR of two keys), $k_8[11, 12]$ and $k_9[12]$. We compute all possible values for the red key bytes by exhaustive search, in time $2^{10\times 8} = 2^{80}$, and store them in a list of size 2^{40}.

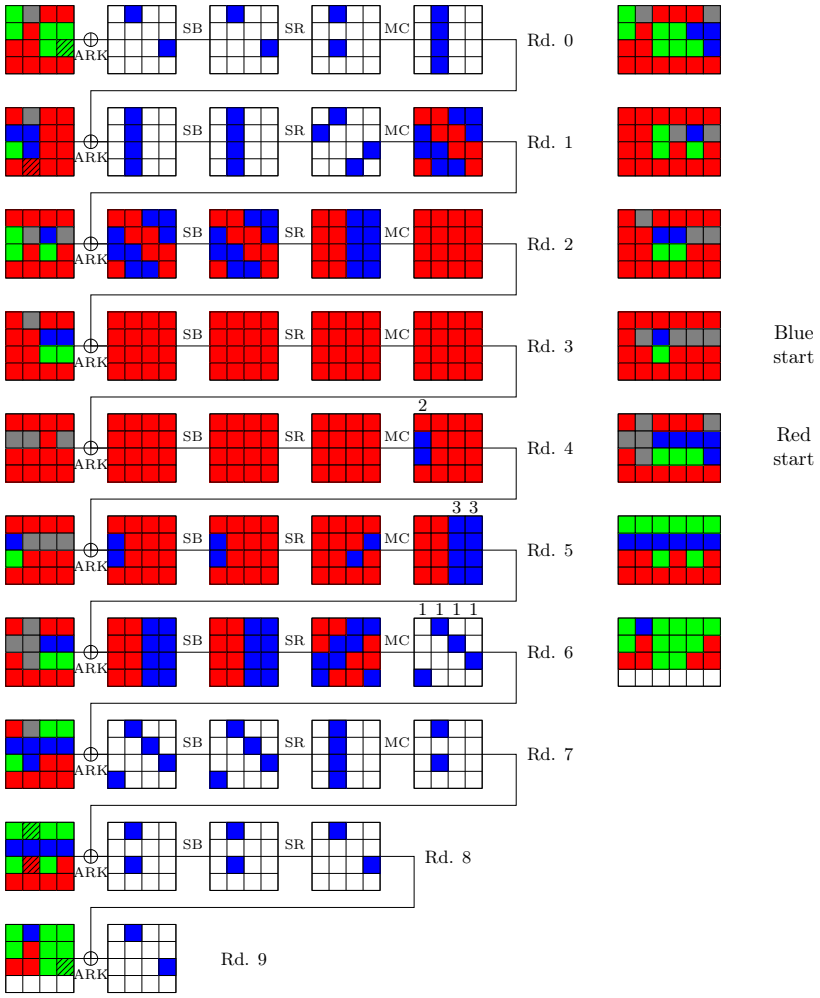
- **Blue key paths:** this is trivial, as there is a single blue byte in the starting state for the blue key. In the key, some of the bytes are obtained by linear combinations of blue and red degrees of freedom, represented in green.
- **Blue list:** we compute forwards, starting from w_3. At round 3, by separating $k_4[11]$ as a linear combination of its red and blue parts, we can write a linear equation of the form:
$$x_4[11] \oplus k_4^b[11] = t = k_4^r[11] \oplus L(z_3[8, 9, 10, 11]) \tag{15}$$
where t is a virtual grey byte. Since $k_4^b[11]$ is known in the blue path, we can deduce $x_4[11]$. At round 4, $w_4[10]$ as well as $z_4[12, 13, 14]$ can be set as grey, allowing to compute forwards until z_5. At round 5, $z_5[0, 1]$ can be set as grey. We need to make one guess per column in columns 1,2,3 to deduce the whole state x_5 using the virtual grey bytes. In the next rounds, there are no virtual bytes, but the cancelations in the red key path remove its influence. Finally we guess $w_0[12, 13]$. In total there are $2^8 \times 2^{5\times 8} = 2^{48}$ choices for the blue path, including 1 degree of freedom from the key and 5 from the state path.
- **Red list:** we compute backwards, starting from z_5. All red bytes in z_5 can be deduced from the virtual grey bytes. Likewise, linear relations allow to deduce new red bytes in z_4 and z_3, and we compute backwards without many any new guesses. In total there are 2^{48} choices for the red path, including 6 degrees of freedom from the key and 0 from the state path.
- **Matching:** There is one linear relation at round 0, two linear relations at round 1, and one linear relation for each column of columns 1,2,3 at round 5, so a total of 6 relations. The merged list has size 2^{48}.

This attack compares favorably to the previous one of [DHS+21] for two reasons. First of all, it improves the memory complexity by a factor 2^8. Second, the path of the attack is arguably simpler than the one in [DHS+21, Figure 22]. Indeed, the attack of [DHS+21] uses complex linear constraints on the key bytes. These linear constraints come from using the alternative representation of the AES key schedule from Leurent and Pernot [LP21], in which one considers linear combinations of the original key bytes. This makes the attack more difficult to represent on a figure. Determining the degrees of freedom in the key schedule and writing down the equations that relate key bytes takes much more effort. Finally, the attack of [DHS+21] also uses *nonlinearly constrained neutral words*, i.e., sub-MITM procedures, which are not necessary for us.

5.4 Simplified Attack on 9-round AES-192

Our model does not find an attack on 10-round AES-192, which is not surprising, as the only known such attack uses the linearization of the AES S-Box [CGL+24]. However, on 9-round AES-192, we are able to recover the best time complexity 2^{112} for pseudo-preimage obtained in [BGST22], with a memory complexity of 2^{112} as well. The path is displayed in Figure 9.

- **Red key paths:** there are 14 red key bytes in the starting state, and later 4 cancelations ($k_1[7]$, $k_0[14] \oplus k_9[14]$, $_8[4, 6]$). The resulting 2^{80} red key paths can be computed in time 2^{112}.



Blue state: 24, Red state: 16, Blue-red state: 14

Blue key: 2, Red key: 14, Grey key: 8

Blue cancelations: 0, Red cancelations: 4

Time: 14, Memory: 14

Total grey: 24, Virtual grey: 12, Loops: 0

Blue list: 14, Red list: 14, Merged list: 14

Figure 9: Pseudo-preimage attack on 9-round AES-192.

- **Blue key paths:** there are only 2 blue key bytes in the starting state, hence 2^{16} paths.
- **Virtual bytes:** there are 2 virtual grey bytes at round 4, which correspond to a linear combination of $x_2[1,2]$ and the blue component of $k_5[1,2]$. There are $3+3$ virtual key bytes at round 5, which are linear combinations of the red bytes in z_5 and the red bytes of k_6. Consider for example column 2, we have a system of 4 independent linear equations:

$$MC^{-1}(x_6[8] \oplus k_6[8], x_6[9] \oplus k_6[9], x_6[10] \oplus k_6^b[10] \oplus k_6^r[10], x_6[11] \oplus k_6[11]) \\ = z_5[8], z_5[9], z_5[10], z_5[11] \ . \quad (16)$$

  We set aside the equation on $z_5[10]$, and we move the expressions in the red key bytes to the right hand side of the system, obtaining three linearly independent equations of the form:

$$\begin{cases} L_1(x_6[8,9,10,11], k_6^b[10]) = t_1 = z_5[8] \oplus L_1'(k_6[8,9,11], k_6^r[10]) \\ L_2(x_6[8,9,10,11], k_6^b[10]) = t_2 = z_5[9] \oplus L_2'(k_6[8,9,11], k_6^r[10]) \\ L_3(x_6[8,9,10,11], k_6^b[10]) = t_3 = z_5[11] \oplus L_3'(k_6[8,9,11], k_6^r[10]) \end{cases} \quad (17)$$

  where t_1, t_2, t_3 are our three virtual grey bytes. Finally, there are 4 virtual grey bytes at round 6.
- **Blue list:** we start at round 4. With the two virtual grey bytes, and the known key bytes, we deduce $x_5[1,2]$ and compute forwards until z_5. From the equations above, we see that by guessing one byte of x_6 for column 2 and 3, we immediately deduce the entire columns.

  At round 6, we use the 4 virtual grey bytes to deduce the 4 blue bytes in w_6. We guess $z_0[5,7]$, compute forwards, and guess $w_1[1,2,6,7,8,11,12,13]$. In total we had to guess 12 state bytes; together with the two key bytes, the list is of size $2^{14\times 8} = 2^{112}$.
- **Red list:** we start at round 6. We need to guess 4 bytes (and deduce the 4 others with virtual grey relations). At z_5 the 3 virtual grey bytes in columns 2 and 3 allow to deduce all new red bytes; similarly at round 4. We compute backwards until w_1. In total we had to guess 4 state bytes, together with the 10 key bytes, the list is of size 2^{112}.
- **Matching:** clearly there are 4 linear relations at round 1, and 8 linear relations at round 2 between red and blue bytes. At round 5 there remains 2 linear relations (from Equation 16) which can be exploited. In total there are 14 relations, so the merged list is of size 2^{112}.

The previous attack on 9-round AES-192 from [BGST22] uses a much more complex technique of *superposition states*, in which keys and internal states are always linear combinations of blue and red degrees of freedom. This makes it more challenging to describe and represent the attack.

6 Applications: New Quantum Attacks

In this section, we give the path of several new quantum MITM pseudo-preimage attacks on AES-based compression functions. The interpretation of the figures, and the computation of the attack parameters, remain the same as in Section 5. The only change is that the paths satisfy the additional constraints of a quantum attack (see Section 3.4): the computation of key paths does not dominate, and there is a quadratic gap between the sizes of the blue and red lists.

6.1 Quantum Attack on 9-round AES-192

Our model finds the first quantum attack on 9-round AES-192, displayed in Figure 10. The memory used is 2^{24} QRAQM, and the time complexity is around 2^{60}, corresponding to a classical attack of complexity 2^{120}.

- **Blue key paths:** there is one blue byte at the starting round, hence 2^8 paths.
- **Red key paths:** there are 9 red bytes at the starting round, and 6 cancelations of red key bytes in the path (3 at round 0 and 9, 3 at round 8). Therefore 2^{24} valid red key paths remain. We find them using Grover's exhaustive search: we need to make 2^{24} iterates before finding a valid path, and since we repeat this 2^{24} times, the total time is 2^{48}.
- **Virtual bytes:** there are 2 virtual bytes at round 4 (all grey). At round 5, $z_5[13, 14]$ may be considered grey, and there is one additional virtual byte in column 2. At round 6 there are 8 virtual bytes.
- **Blue list:** we start at round 4, and deduce the two blue bytes using virtual grey relations. At round 5 we deduce the 3 new blue bytes using virtual grey relations. At round 6 we deduce the 8 blue bytes in w_6 using the 8 virtual grey relations, so no guesses had to be made until now. We compute forwards until z_1. We guess $x_1[1, 2]$. In total the blue list has size 2^{24}.
- **Red list:** we start at round 6. We need to guess 3 bytes (one in columns 0, 2 and 3) and deduce all others in z_6 using virtual grey relations. At round 5 we also have enough virtual grey to not make any new guess. We compute until round 1. Thus the red list size is $2^{24} \times 2^{24} = 2^{48}$.
- **Matching:** there is one relation at round 1 and two relations at round 2, thus the merged list size is 2^{48}.

As there are 33 grey bytes in total (incl. 13 virtual), we need to repeat the merging for $2^{(33-24)\times 8} = 2^{72}$ choices of virtual grey bytes in order to find a solution. The quantum time complexity is therefore: $2^{72/2} \times \left(2^{24} + 2^{48/2}\right) = 2^{60}$.

6.2 Quantum Attack on 7-round AES-128

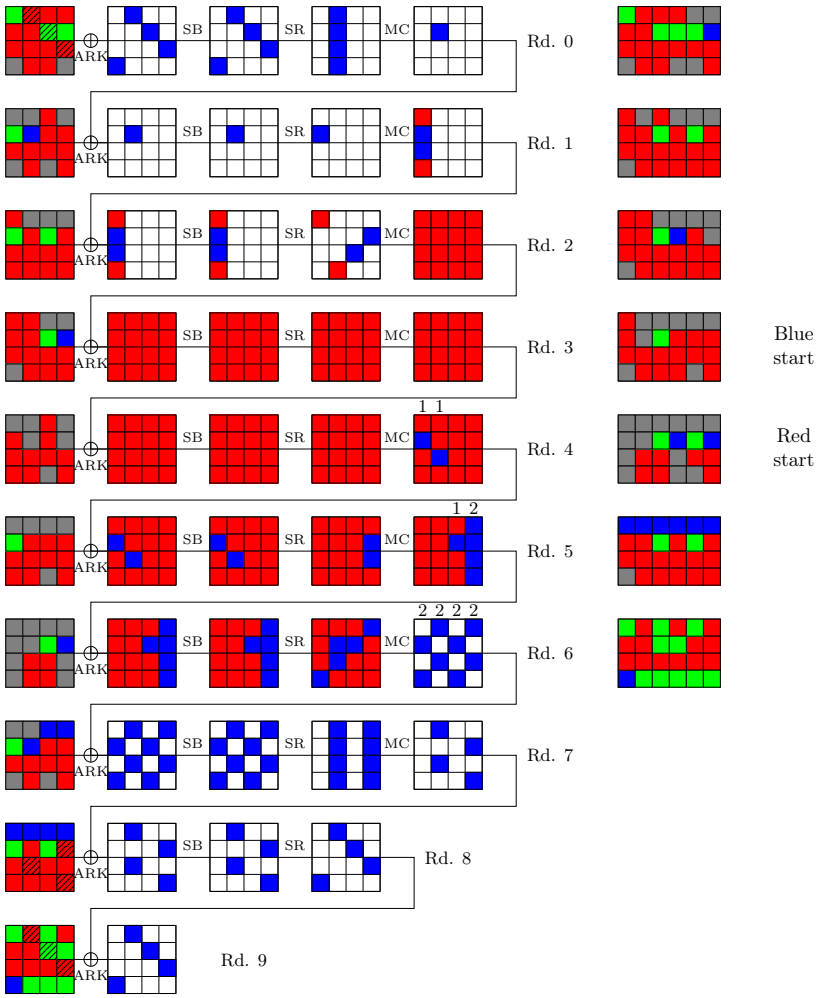
Our model finds an improved quantum attack on 7-round AES-128, displayed in Figure 10. The memory used is 2^{16} QRAQM, and the time complexity is around 2^{56}, corresponding to a classical attack of complexity 2^{112}.

There are 2 blue bytes in the key and 12 virtual grey bytes. The blue list has size 2^{16}, as there are no degrees of freedom from the state: all blue bytes can be deduced from the key and the virtual grey values. The red list has size 2^{32}, where these 4 degrees of freedom are gained at round 4 (1 in column 1, 2 in column 2, 1 in column 3). There are two matching points at round 1. As there are 26 grey bytes in total (incl. 12 virtual), we need to repeat the merging for $2^{(26-16)\times 8} = 2^{80}$ values of the virtual grey bytes in order to find a solution. The quantum time complexity is therefore $2^{80/2}(2^{16} + 2^{32/2}) = 2^{56}$.

6.3 Quantum Attack on 9-round AES-256

Our model finds the first quantum attack on 9-round AES-256, displayed in Figure 12. The memory used is 2^8 QRAQM.

There are 2 red bytes in the key and one cancelation constraint, so 2^8 valid red key paths in total. The red list has size 2^8, as all red bytes in the state can be computed using the virtual grey values and the red key bytes. The blue list has size 2^{16}, which also comes



Quantum attack

Blue state: 15, Red state: 16, Blue-red state: 15

Blue key: 1, Red key: 9, Grey key: 14

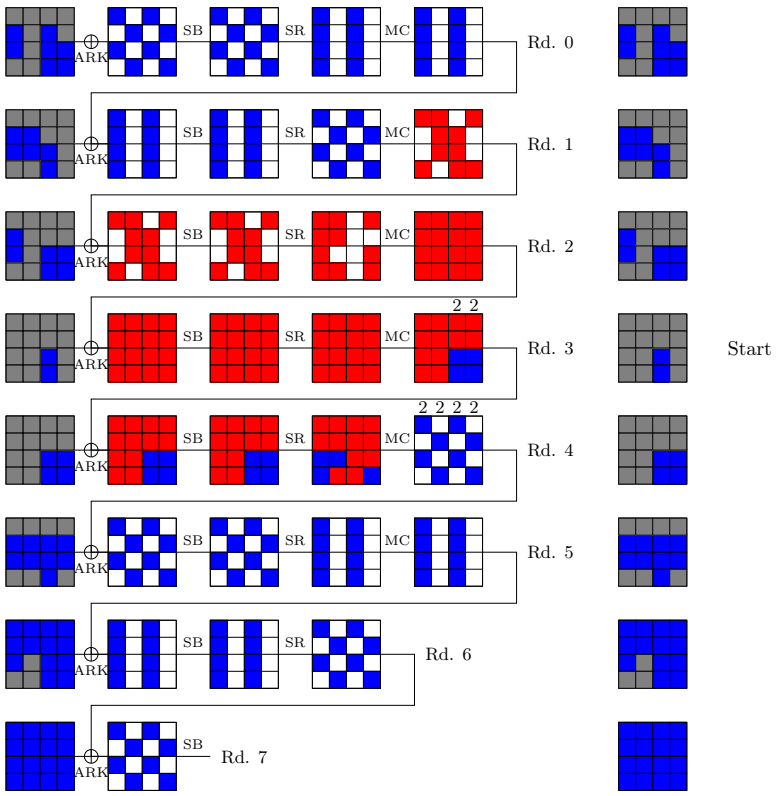
Blue cancelations: 0, Red cancelations: 6

Time: 15, Memory: 3

Total grey: 33, Virtual grey: 13, Loops: 9

Blue list: 3, Red list: 6, Merged list: 6

Figure 10: Quantum pseudo-preimage attack on 9-round AES-192.



Quantum attack

Blue state: 12, Red state: 16, Blue-red state: 14

Blue key: 2, Red key: 0, Grey key: 14

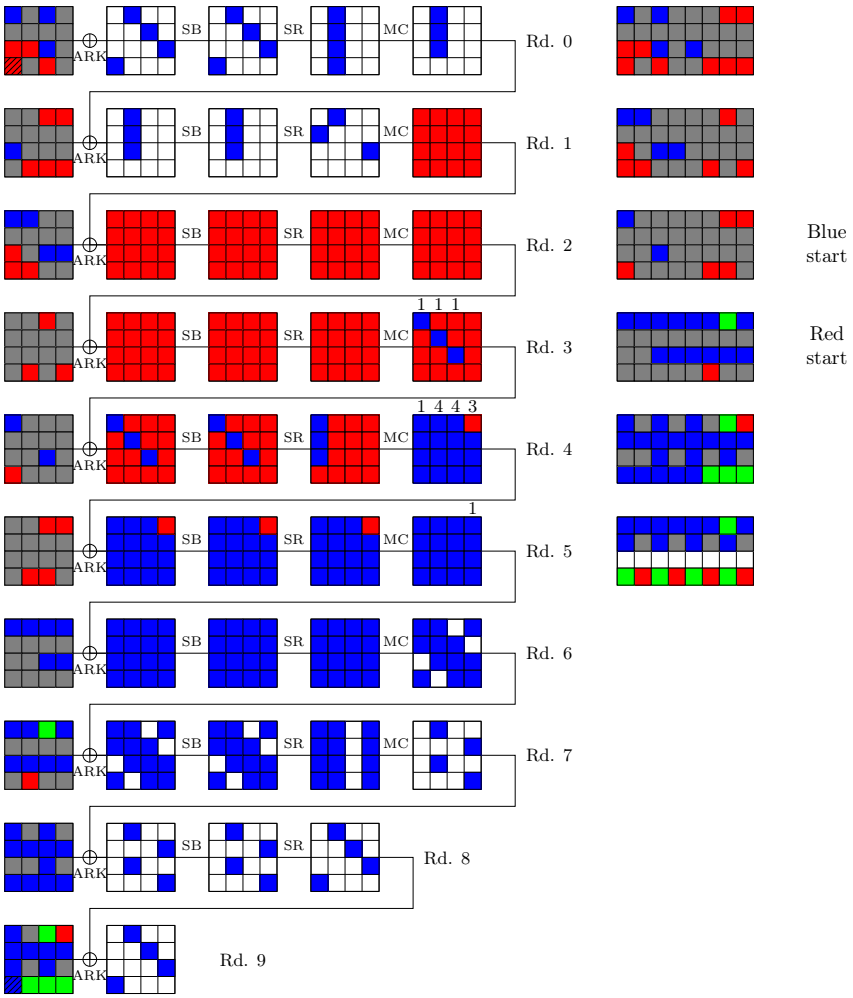
Blue cancelations: 0, Red cancelations: 0

Time: 14, Memory: 2

Total grey: 26, Virtual grey: 12, Loops: 10

Blue list: 2, Red list: 4, Merged list: 4

Figure 11: Quantum pseudo-preimage attack on 7-round AES-128.



Quantum attack

Blue state: 16, Red state: 16, Blue-red state: 13

Blue key: 2, Red key: 2, Grey key: 28

Blue cancelations: 0, Red cancelations: 1

Time: 15, Memory: 1

Total grey: 45, Virtual grey: 16, Loops: 13

Blue list: 2, Red list: 1, Merged list: 0

Figure 12: Quantum pseudo-preimage attack on 9-round AES-256.

from the key only.There are 16 virtual grey and 45 grey bytes in total. We need to repeat the merging for $2^{(45-32)\times 8} = 2^{104}$ values of the virtual grey bytes, which eventually gives a complexity 2^{60}.

6.4 Quantum Attack on 6-round Whirlpool

We obtain the first attack against 6-round `Whirlpool`, displayed in Figure 13. Note that we have replaced the ShiftColumns (resp. MixRows) of the original design by ShiftRows (resp. MixColumns), as this is equivalent up to a rotation of the internal state and key. Thus the figure will follow the convention of [BGST22, CGL+24].

The path of the attack is quite close to the 6-round classical attack of [BGST22], but as a classical attack, its time complexity is slightly higher (2^{464}) and the memory complexity is lower (2^{128}). In the quantum setting, the memory complexity is around 2^{128} QRAQM and the time complexity is around 2^{232}. Contrary to all our other attacks, this one switches between key additions made with the round key k or the equivalent round key u. This is because of the specific key schedule of `Whirlpool`, which uses the same MC operation as the state path; switching allows more sparse patterns of blue bytes in the round keys.

7 Conclusion

In this paper, we present a simple yet accurate model for MITM preimage attacks against compression functions based on `AES`-based designs. Our approach offers a key advantage: it establishes a direct correspondence between the solution of the MILP model and the associated attack, ensuring that the attack is always valid. While we successfully recover previously known best attacks on the three different versions of `AES`, we also improved some of their memory complexities and found new quantum attacks.

The technicality of prior models was a challenge already discussed, e.g., in [BGST22]. We believe our approach to be more accessible. However, it still suffers from scalability issues: solving the models without any hints on the color patterns can sometimes be impractical, in the sense that we are not able to prove the optimality of the attacks. Another interesting question would be to incorporate S-Box linearization like in [CGL+24]. Finally, it would be useful to extend our approach beyond `AES`-based primitives to other cryptographic designs.

Acknowledgements. This work has been supported by the French Agence Nationale de la Recherche through the QATS project under Contract ANR-24-CE39-7894-01, through the OREO project under Contract ANR-22-CE39-0015, and through the France 2030 program under grant agreement No. ANR-22-PECY-0010 CRYPTANALYSE.

References

[AS08] Kazumaro Aoki and Yu Sasaki. Preimage attacks on one-block MD4, 63-step MD5 and more. In *Selected Areas in Cryptography*, volume 5381 of *LNCS*, pages 103–119. Springer, 2008. doi:10.1007/978-3-642-04159-4_7.

[BDF11] Charles Bouillaguet, Patrick Derbez, and Pierre-Alain Fouque. Automatic search of attacks on round-reduced AES and applications. In *CRYPTO*, volume 6841 of *Lecture Notes in Computer Science*, pages 169–187. Springer, 2011. doi:10.1007/978-3-642-22792-9_10.

[BDG+19] Zhenzhen Bao, Lin Ding, Jian Guo, Haoyang Wang, and Wenying Zhang. Improved meet-in-the-middle preimage attacks against AES hashing modes.

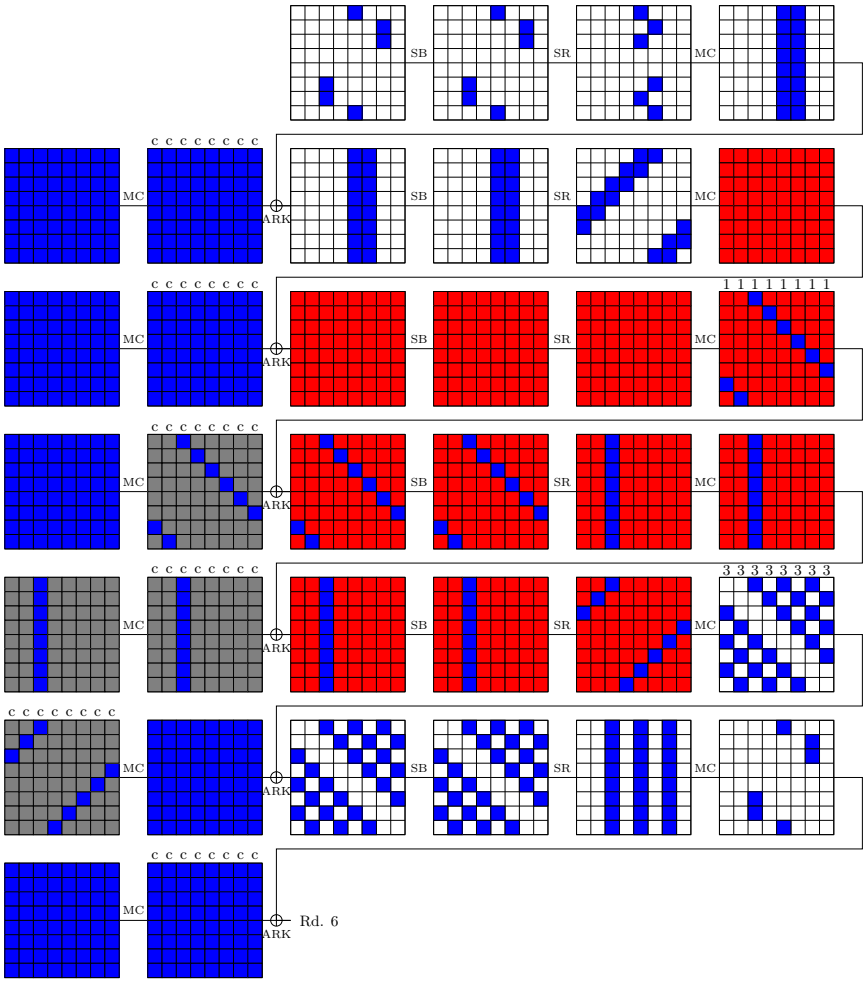


Figure 13: Quantum pseudo-preimage attack on 6-round `Whirlpool`. "c" on the figure indicates a choice between the column of k or the corresponding $u = MC^{-1}(k)$ for key addition.

IACR Trans. Symmetric Cryptol., 2019(4):318–347, 2019. doi:10.13154/tosc.v2019.i4.318-347.

[BDG+21] Zhenzhen Bao, Xiaoyang Dong, Jian Guo, Zheng Li, Danping Shi, Siwei Sun, and Xiaoyun Wang. Automatic search of meet-in-the-middle preimage attacks on AES-like hashing. In *EUROCRYPT (1)*, volume 12696 of *LNCS*, pages 771–804. Springer, 2021. doi:10.1007/978-3-030-77870-5_27.

[BGST22] Zhenzhen Bao, Jian Guo, Danping Shi, and Yi Tu. Superposition meet-in-the-middle attacks: Updates on fundamental security of AES-like hashing. In *CRYPTO (1)*, volume 13507 of *Lecture Notes in Computer Science*, pages 64–93. Springer, 2022. doi:10.1007/978-3-031-15802-5_3.

[BHMT02] Gilles Brassard, Peter Hoyer, Michele Mosca, and Alain Tapp. Quantum amplitude amplification and estimation. *Contemporary Mathematics*, 305:53–74, 2002.

[BR10] Andrey Bogdanov and Christian Rechberger. A 3-subset meet-in-the-middle attack: Cryptanalysis of the lightweight block cipher KTANTAN. In *Selected Areas in Cryptography*, volume 6544 of *LNCS*, pages 229–240. Springer, 2010. doi:10.1007/978-3-642-19574-7_16.

[CGL+24] Shiyao Chen, Jian Guo, Eik List, Danping Shi, and Tianyu Zhang. Diving deep into the preimage security of AES-like hashing. In *EUROCRYPT (1)*, volume 14651 of *Lecture Notes in Computer Science*, pages 398–426. Springer, 2024. doi:10.1007/978-3-031-58716-0_14.

[CGL+25] Shiyao Chen, Jian Guo, Eik List, Danping Shi, and Tianyu Zhang. Scrutinizing the security of aes-based hashing and one-way functions. *IACR Cryptol. ePrint Arch.*, page 792, 2025. URL: https://eprint.iacr.org/2025/792.

[CNV13] Anne Canteaut, María Naya-Plasencia, and Bastien Vayssière. Sieve-in-the-middle: Improved MITM attacks. In *CRYPTO (1)*, volume 8042 of *LNCS*, pages 222–240. Springer, 2013. doi:10.1007/978-3-642-40041-4_13.

[DF16] Patrick Derbez and Pierre-Alain Fouque. Automatic search of meet-in-the-middle and impossible differential attacks. In *CRYPTO (2)*, volume 9815 of *Lecture Notes in Computer Science*, pages 157–184. Springer, 2016. doi:10.1007/978-3-662-53008-5_6.

[DH77] Whitfield Diffie and Martin E. Hellman. Special feature exhaustive cryptanalysis of the NBS data encryption standard. *Computer*, 10(6):74–84, 1977. doi:10.1109/C-M.1977.217750.

[DHS+21] Xiaoyang Dong, Jialiang Hua, Siwei Sun, Zheng Li, Xiaoyun Wang, and Lei Hu. Meet-in-the-middle attacks revisited: Key-recovery, collision, and preimage attacks. In *CRYPTO (3)*, volume 12827 of *LNCS*, pages 278–308. Springer, 2021. doi:10.1007/978-3-030-84252-9_10.

[DR20] Joan Daemen and Vincent Rijmen. *The Design of Rijndael - The Advanced Encryption Standard (AES), Second Edition.* Information Security and Cryptography. Springer, 2020. doi:10.1007/978-3-662-60769-5.

[DSP07] Orr Dunkelman, Gautham Sekar, and Bart Preneel. Improved meet-in-the-middle attacks on reduced-round DES. In *INDOCRYPT*, volume 4859 of *LNCS*, pages 86–100. Springer, 2007. doi:10.1007/978-3-540-77026-8_8.

[DZQ+24] Xiaoyang Dong, Boxin Zhao, Lingyue Qin, Qingliang Hou, Shun Zhang, and Xiaoyun Wang. Generic mitm attack frameworks on sponge constructions. In *CRYPTO (4)*, volume 14923 of *Lecture Notes in Computer Science*, pages 3–37. Springer, 2024. doi:10.1007/978-3-031-68385-5_1.

[GLRW10] Jian Guo, San Ling, Christian Rechberger, and Huaxiong Wang. Advanced meet-in-the-middle preimage attacks: First results on full tiger, and improved results on MD4 and SHA-2. In *ASIACRYPT*, volume 6477 of *LNCS*, pages 56–75. Springer, 2010. doi:10.1007/978-3-642-17373-8_4.

[Gro96] Lov K. Grover. A fast quantum mechanical algorithm for database search. In *STOC*, pages 212–219. ACM, 1996. doi:10.1145/237814.237866.

[HDQ+23] Qingliang Hou, Xiaoyang Dong, Lingyue Qin, Guoyan Zhang, and Xiaoyun Wang. Automated meet-in-the-middle attack goes to feistel. In *ASIACRYPT (3)*, volume 14440 of *Lecture Notes in Computer Science*, pages 370–404. Springer, 2023. doi:10.1007/978-981-99-8727-6_13.

[HDS+22] Jialiang Hua, Xiaoyang Dong, Siwei Sun, Zhiyu Zhang, Lei Hu, and Xiaoyun Wang. Improved MITM cryptanalysis on streebog. *IACR Trans. Symmetric Cryptol.*, 2022(2):63–91, 2022. doi:10.46586/TOSC.V2022.I2.63-91.

[KRS12] Dmitry Khovratovich, Christian Rechberger, and Alexandra Savelieva. Bicliques for preimages: Attacks on skein-512 and the SHA-2 family. In *FSE*, volume 7549 of *LNCS*, pages 244–263. Springer, 2012. doi:10.1007/978-3-642-34047-5_15.

[LP21] Gaëtan Leurent and Clara Pernot. New representations of the AES key schedule. In *EUROCRYPT (1)*, volume 12696 of *Lecture Notes in Computer Science*, pages 54–84. Springer, 2021. doi:10.1007/978-3-030-77870-5_3.

[LWZ16] Li Lin, Wenling Wu, and Yafei Zheng. Automatic search for key-bridging technique: Applications to lblock and TWINE. In *FSE*, volume 9783 of *Lecture Notes in Computer Science*, pages 247–267. Springer, 2016. doi:10.1007/978-3-662-52993-5_13.

[NC02] Michael A Nielsen and Isaac Chuang. Quantum computation and quantum information, 2002.

[QHD+23] Lingyue Qin, Jialiang Hua, Xiaoyang Dong, Hailun Yan, and Xiaoyun Wang. Meet-in-the-middle preimage attacks on sponge-based hashing. In *EUROCRYPT (4)*, volume 14007 of *Lecture Notes in Computer Science*, pages 158–188. Springer, 2023. doi:10.1007/978-3-031-30634-1_6.

[Sas11] Yu Sasaki. Meet-in-the-middle preimage attacks on AES hashing modes and an application to Whirlpool. In *FSE*, volume 6733 of *LNCS*, pages 378–396. Springer, 2011. doi:10.1007/978-3-642-21702-9_22.

[Sas18] Yu Sasaki. Integer linear programming for three-subset meet-in-the-middle attacks: Application to GIFT. In *IWSEC*, volume 11049 of *LNCS*, pages 227–243. Springer, 2018. doi:10.1007/978-3-319-97916-8_15.

[SS22] André Schrottenloher and Marc Stevens. Simplified MITM modeling for permutations: New (quantum) attacks. In *CRYPTO (3)*, volume 13509 of *Lecture Notes in Computer Science*, pages 717–747. Springer, 2022. doi:10.1007/978-3-031-15982-4_24.

[SS23] André Schrottenloher and Marc Stevens. Simplified modeling of MITM attacks for block ciphers: New (quantum) attacks. *IACR Trans. Symmetric Cryptol.*, 2023(3):146–183, 2023. doi:10.46586/TOSC.V2023.I3.146-183.

[SSD+18] Danping Shi, Siwei Sun, Patrick Derbez, Yosuke Todo, Bing Sun, and Lei Hu. Programming the demirci-selçuk meet-in-the-middle attack with constraints. In *ASIACRYPT (2)*, volume 11273 of *Lecture Notes in Computer Science*, pages 3–34. Springer, 2018. doi:10.1007/978-3-030-03329-3_1.